Strojové učení a rozpoznávání (SUR)

Person detector based on audio record and face image

Matúš Moravčík (xmorav48)

May 2026

1 System Architecture

The system consists of two independent unimodal classifiers – one operating on face images and one on voice recordings – whose probability scores are combined by a score-level fusion model. Figure 1 gives an overview of the full pipeline.

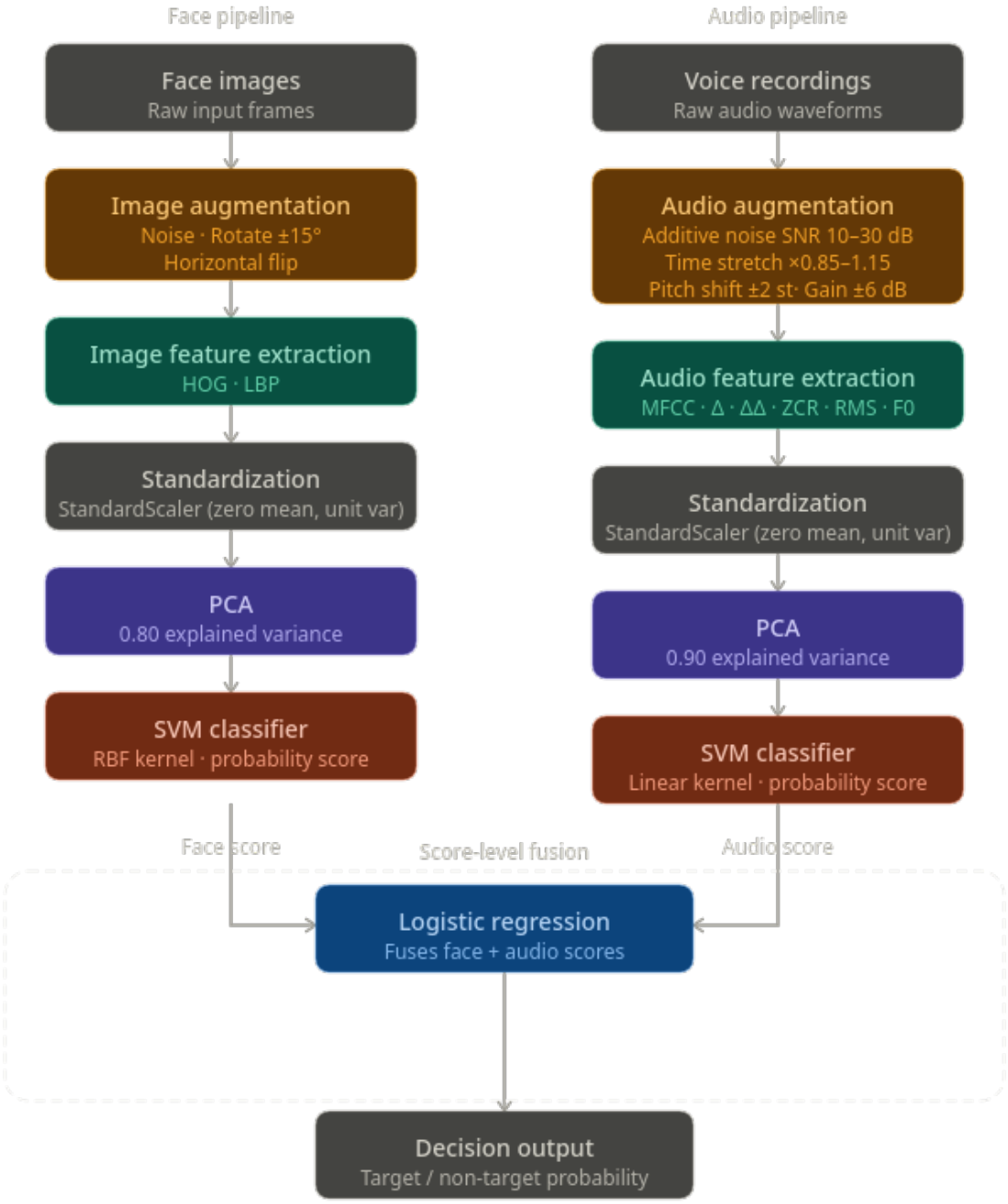

Figure 1: Full bimodal pipeline. Both modalities are independently processed through feature extraction, standardization, PCA, and an SVM classifier. Their output probability scores are fed into a logistic regression fusion model.

Design rationale. The dataset is small (fewer than 200 unique recordings per modality before augmentation) and highly imbalanced (approximately 1:6 target-to-non-target ratio). These constraints ruled out deep learning models.

1.1 Configuration

All hyper-parameters – number of augmentation samples, PCA variance threshold, SVM kernel type, regularisation constant C, and decision threshold – are stored in a single `config.yaml` file. The project uses `hydra-core` for configuration loading, so any parameter can be overridden from the command line without editing source files, e.g.:

```
python train.py model.image.C=0.05 augmentation.n_copies=5
```

1.2 Cross-validation Strategy

Because the target class contains only one individual, person-stratified splitting is not meaningful. Instead, splits are formed by *recording session*: all samples that share the same session identifier (the second underscore-delimited field in the filename, e.g. `f401_01_...`) are kept together in a single fold. This prevents information leakage caused by shared recording conditions such as background noise, microphone characteristics, or lighting.

The target class contains three distinct sessions, so we used $k = 3$ folds, placing one session in the validation set at a time. Non-target data were distributed across folds proportionally. Using more than three folds would have required placing all target sessions in training data simultaneously, making unbiased evaluation impossible.

2 Data Augmentation

The limited size of the dataset makes overfitting the primary risk. Augmentation serves two purposes: (1) increasing the effective number of training samples and (2) exposing the model to realistic variations that may appear in the unseen evaluation data (noise, reverberation, lighting changes).

For each original training sample we generated **three** augmented copies. Generating more copies per sample risks flooding the dataset with near-identical examples that provide no additional information, while the augmentations used are already relatively mild.

2.1 Audio Augmentation

Each augmentation pass applies the following transforms independently and stochastically:

- **Additive Gaussian noise** (60 % probability): white noise is mixed at a randomly sampled SNR between 10 and 30 dB, simulating different recording environments.
- **Time stretching** (50 % probability): the playback rate is changed by a factor uniformly drawn from $[0.85, 1.15]$. This models natural variation in speaking rate without distorting pitch.
- **Pitch shifting** (40 % probability): the pitch is shifted by ± 2 semitones, accounting for within-speaker pitch variation across sessions.
- **Random gain** (50 % probability): a gain between -6 and $+6$ dB is applied, simulating different microphone gains and distances.

Each transform is applied independently of the others, so multiple transforms can co-occur in a single augmented sample.

2.2 Image Augmentation

Image augmentation performs combination up to three transforms per copy:

- **Gaussian noise** (50 % weight): zero-mean noise with $\sigma = 0.05$ is added to the normalised pixel values and clipped to $[0, 1]$.
- **Random rotation** (50 % weight): the image is rotated by a uniformly sampled angle in $[-15, 15]$ with constant-fill padding.

- **Horizontal flip** (60 % weight): the image is mirrored left-to-right. Because facial identity is symmetric, this is a particularly safe augmentation.

3 Feature Extraction

3.1 Audio Features

We extract a concatenated feature vector from each voice recording:

- **MFCCs** (13 coefficients): capture the spectral envelope, which encodes vocal-tract shape and is the standard speaker-discriminative feature set.
- **Delta and delta-delta MFCCs**: first- and second-order temporal derivatives encode the dynamics of articulation, which differ between speakers even when the static spectrum is similar.
- **Zero-crossing rate (ZCR)**: a simple measure of signal noisiness and voicing that carries complementary speaker information.
- **Root mean square energy (RMS)**: captures speaker-level loudness patterns.
- **Fundamental frequency (F_0)**: pitch contour statistics encode prosodic characteristics that are speaker-specific.

Frame-level features are summarised by their mean and standard deviation across the recording, yielding a fixed-length vector of approximately 250 dimensions. We chose handcrafted acoustic features over raw waveform representations because the dataset is too small to learn useful representations from scratch.

3.2 Image Features

Face images are described using two complementary texture descriptors:

- **Histogram of Oriented Gradients (HOG)**: captures edge structure and local shape, making it robust to moderate changes in illumination and scale.
- **Local Binary Patterns (LBP)**: encode micro-texture at a per-pixel level; highly resistant to monotonic illumination changes.

The concatenation of both descriptors produces a vector of over 2 000 dimensions. Together, HOG and LBP cover complementary aspects of facial appearance: coarse shape and fine texture. Deep features were not considered because the project rules forbid pre-trained models and training a CNN from scratch on fewer than 600 images would invariably overfit.

4 Models

4.1 Standardization and Dimensionality Reduction

Before classification, all feature vectors are standardized to zero mean and unit variance. Standardization is necessary because (i) PCA is sensitive to scale, and (ii) the extracted features span very different numerical ranges (e.g. RMS energy vs. LBP histogram bins).

PCA is then applied to reduce dimensionality. With only 592 image training samples (including augmentation) against 2 000+ raw features, the curse of dimensionality would render any linear model unreliable without reduction. We target a compressed space of roughly 100 dimensions in both modalities by setting the `n_components` of PCA to the fraction of explained variance that achieves this:

- **Image pipeline**: 0.80 explained variance
- **Audio pipeline**: 0.95 explained variance

The higher threshold for audio reflects the fact that the raw audio feature space is smaller (∼250 dimensions), so a larger fraction of variance must be retained to arrive at a comparable compressed size.

4.2 Classifiers

Image – SVM (RBF kernel, $C = 0.1$). An SVM with probability calibration (`probability=True`, `class_weight="balanced"`) is used for face classification. SVMs generalise well on small, high-dimensional data because the decision boundary depends only on support vectors. The RBF kernel captures non-linear structure in the face-feature space; C was reduced from the default 1.0 to 0.1 to counteract overfitting observed during cross-validation (training AUC ≈1.0 at higher C). An MLP was tested but yielded lower validation AUC due to overfitting despite dropout.

Audio – SVM. A Support Vector Machine with a linear kernel is used for the audio pipeline. Prior to classification, PCA is applied to retain 95% of the variance, reducing feature dimensionality and mitigating noise. A relatively small regularization parameter ($C = 0.01$) is chosen to control model complexity and improve generalization. The linear kernel was preferred over non-linear alternatives due to its stability and lower risk of overfitting on the available audio data. A fixed decision threshold of 0.2 is used to produce final predictions.

4.3 Score-level Fusion

The probability outputs of both unimodal models are concatenated into a score vector $[p_{\text{image}}, p_{\text{audio}}]$ and passed to a logistic regression classifier, which learns a weighted combination and applies a sigmoid to produce the final target probability. The learned log-odds weights (image: 6.87440684, audio: 8.5777156) reflect the differing score scales of the two models rather than equal reliability; in practice the audio model slightly dominates the final decision. The fusion model is trained within the same cross-validation folds to avoid leakage.

5 Evaluation Metrics

Primary metric – AUC-ROC measures ranking ability independently of the decision threshold and is robust to class imbalance. **Secondary metric – F1** is reported at a fixed threshold $\tau = 0.20$. The low threshold compensates for the 1:6 class imbalance: empirically, the optimal threshold on training folds (≈0.50) was much higher than on validation folds (≈0.15), indicating over-confidence on training data.

Table 1: 3-fold cross-validated performance (mean ± std). F1 evaluated at $\tau = 0.20$.

Model	AUC-ROC	Best AUC	F1 @ τ=0.20	Best F1
Image SVM (RBF)	0.9964 ± 0.0041	1.0000	0.8111 ± 0.2006	1.0000
Audio SVM (linear)	0.9823 ± 0.0135	1.0000	0.8769 ± 0.0782	0.9524
Fusion (LR)	0.9927 ± 0.0103	1.0000	0.8410 ± 0.1243	0.9474
Audio GMM†	0.9573 ± 0.0182	0.9766	0.7220 ± 0.0950	0.8000

†Previous model, replaced by SVM in the final system.

Figures 2 and 3 show the average confusion matrices and DET curves over the three validation folds. Figure 4 shows qualitative predictions on the evaluation face images.

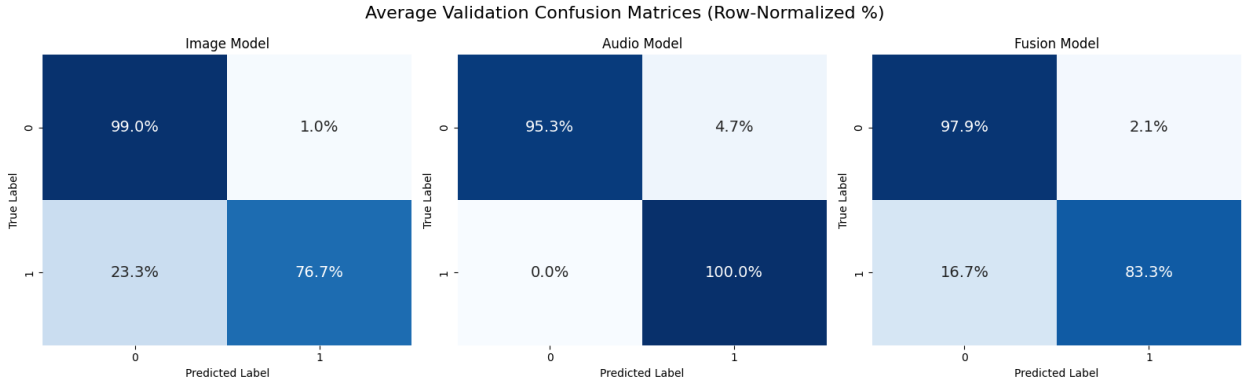

Figure 2: Average normalised confusion matrices at $\tau = 0.20$ over the three cross-validation folds.

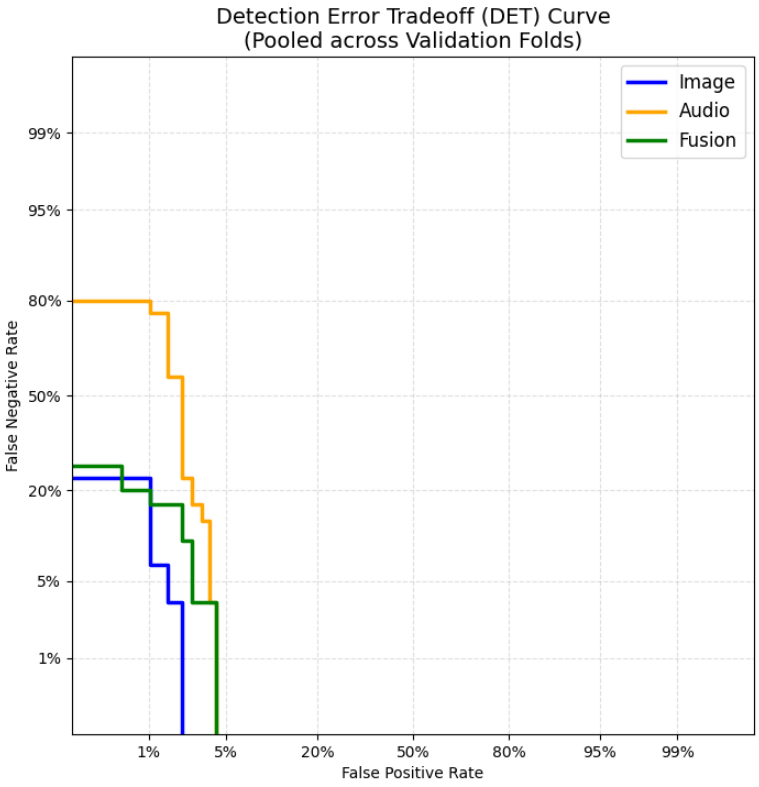

Figure 3: Detection Error Trade-off (DET) curves averaged over the three validation folds. Lower curves indicate better performance.

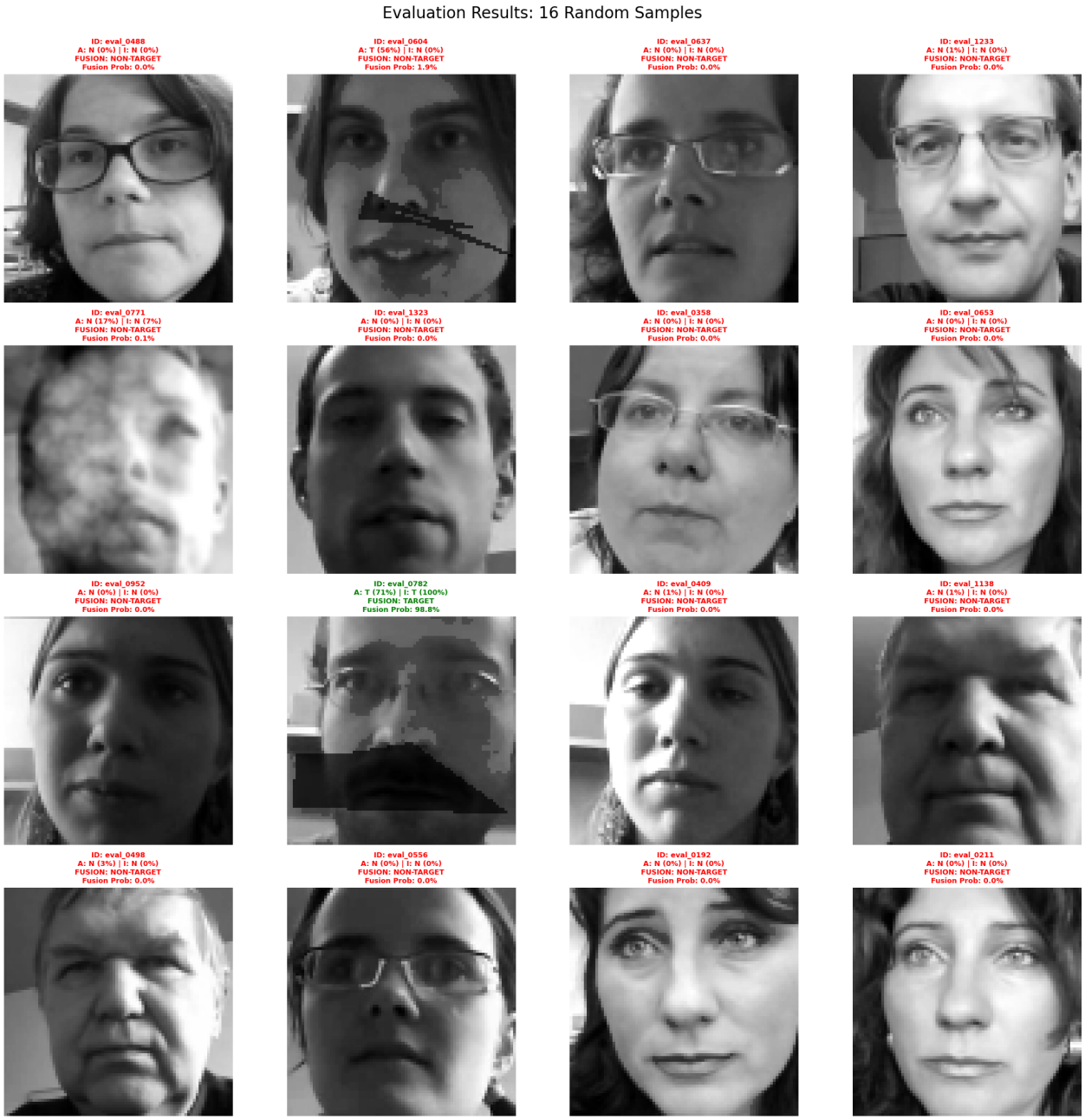

Figure 4: Qualitative evaluation output. Each face is annotated with the predicted probability from the image model, audio model, and fusion model, together with the final hard label (1 = target, 0 = non-target) produced by the fusion model at $\tau = 0.50$.

6 Regularization and Generalization

The following measures were applied to improve generalization:

Dimensionality reduction. PCA reduces the feature space to ∼100 components, dramatically lowering the effective number of free parameters the SVM must fit.

SVM regularization parameter C. C controls the trade-off between margin width and training error. Reducing C from the default value of 1.0 to 0.1 (image) and 0.01 (audio) encourages wider margins and smoother decision boundaries. The values were selected by grid search over the three cross-validation folds.

Kernel choice. For the audio pipeline, switching from RBF to a linear kernel was the single most

effective regularization step: the linear kernel has fewer implicit parameters and cannot produce the arbitrarily curved boundaries that caused the RBF model to memorise training sessions.

Data augmentation. As described in Section 2, augmentation increases sample diversity, effectively acting as implicit regularization by preventing the model from exploiting spurious correlations present only in specific recording sessions (e.g. identical background noise).

Session-based cross-validation. Using recording sessions as split boundaries ensures that validation performance reflects generalization to new sessions, not merely new files from the same session. This provides an honest estimate of performance on the unseen evaluation data, where session conditions will differ from anything seen during training.

7 Reproducibility

7.1 Dependencies

```
Python = 3.12 + requirements.txt
```

7.2 Installation

```
# Create and activate virtual environment
python3.12 -m venv env
source ./env/bin/activate

# Install dependencies
pip install -r requirements.txt
pip install -e .
```

7.3 Running the System

```
# Train all models and run cross-validation
python src/train.py

# Generate score files for the evaluation data
python src/predict.py paths.eval_dir=<path_to_eval_data>

# Output files (1 for TARGET, 0 for NON-TARGET) are written to
#   audio_SVM.txt
#   image_SVM.txt
#   fusion_Late_LR.txt
```